Project Report

IMT2022001 Siddharth Menon

November 27, 2025

Abstract

In this project, we execute two kinds of queries on call data generated using a script.

1 Dataset

The dataset `mono_signal.csv` contains the following fields:

- unique_id
- start_ts
- end_ts
- caller
- callee
- disposition
- imei
- id

Example record:

```
15785,1764079982000,1764079984000,9444783547,9449869684,
connected,108228093,0
```

The dataset `bi_signal.csv` contains the following fields:

- unique_id
- event_type (denoting call start/end)
- timestamp
- caller
- callee
- disposition

- imei
- id

Example record:

```
15785,0,1764079984000,9444783547,9449869684,
connected,108228093,0
```

The record was generated using the generate.py script (included in the GitHub repo), with throughput, total duration and max span specified as arguments.

2 Methodology

2.1 Loading the CSV in Spark

In order to load in this pre-generated data in the form of a stream into Spark, we use Spark's inbuilt RateStreamSource, which generates numbers (0,1,2...) with timestamps at a specified throughput. We then join this data with the pre-generated data, also equipped with an id field, to effectively achieve the same throughput as the RateStreamSource.

```
df = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("mono_signal.csv")

df.createOrReplaceTempView("mono_signal")

rate_df = (
    spark.readStream
        .format("rate")
        .option("rowsPerSecond", 10000)
        .option("rampUpTime", 0)
        .load()
)

joined_df = (
    stream_with_id
        .join(csv_df, on="id", how="left")
        .select("timestamp", "value", "id", "rate_ts", *csv_df.
           columns)
)
```

2.2 SQL Query

SQL Querying was done for sanity checks and verifying results later on.

First Query:

Finding the number of active calls in 30 second windows. The GitHub repository with the benchmark and queries had a slight inconsistency here, because although the question asked us to find the total active calls in 30 second windows, the sample query found the calls that started in 30 second windows.

We ended up doing it for both interpretations of the query.

For the first interpretation:

```
WITH Expanded AS (
  SELECT
    unique_id,
    start_ts,
    end_ts,
    generate_series(
      (start_ts / 30000) * 30000,
      (end_ts   / 30000) * 30000,
      30000
    )::bigint AS WindowStart
  FROM mono_signal
)
SELECT
  WindowStart,
  COUNT(DISTINCT(unique_id)) AS ActiveCalls
FROM Expanded
GROUP BY WindowStart
ORDER BY WindowStart;
```

For the second interpretation:

```
SELECT
  FLOOR(start_ts / 300000) AS WindowID,
  COUNT(*) AS ActiveCalls
FROM mono_signal
GROUP BY WindowID
ORDER BY WindowID;
```

Second Query:

Finding consecutive calls by the same caller which had a gap of at least 60 seconds between them.

```
SELECT
    a.caller,
```

```
    a.end_ts   AS PrevEnd,
    b.start_ts AS NextStart,
    b.start_ts - a.end_ts AS IdleGap
FROM mono_signal a
JOIN mono_signal b
    ON  b.caller = a.caller
    AND b.start_ts > a.end_ts
WHERE NOT EXISTS (
    SELECT 1
    FROM mono_signal c
    WHERE c.caller = a.caller
      AND c.start_ts > a.end_ts
      AND c.start_ts < b.start_ts
)
AND b.start_ts - a.end_ts > 60000
ORDER BY a.caller, a.end_ts;
```

3 Query Execution and Spark Architecture

3.1 Query 1

3.1.1 Mono Signal

The major challenge faced here was that the events only arrive at their event end time. As an implication of this, apart from a lateness watermark (which was taken as 1 second), we also needed another watermark which is as much as the maximum span. This is due to the chance that events belonging to a window may arrive after the actual end time of the window, because it probably started at a time near the end of the window, but got over and arrived after the window end time. Hence, we needed to delay our window release by as much as the maximum span. This also reflects in our query latency.

3.1.2 Bi Signal

Bi-Signal was relatively easier to deal with. This is due to the fact that we no longer need to maintain two watermarks, and can revert to only having one, for late events. Although the arrival of 2 events for a single record doubles the throughput, we still see a massive drop in latency, because we can release results immediately at the window end, by virtue of us already having received the start signal for a record.

3.2 Query 2

Query 2 was not a windowed query at all, and needs the entire data before we can query it. This is owing to the nature of the query itself, as it is possible that two consecutive calls may be towards either end of the entire duration. Hence, we cannot maintain a watermark which would allow us to prune old records from memory.

Trying to execute this as a streaming query led to the JVM heap memory being depleted, and the program crashing as a result. Attempts to increase allocation were also unsuccessful.

As a last resort, this query was treated as a static query, with only one query being run in the end.

4 Results

4.1 Query 1:

4.1.1 Effect on increasing throughput

:

Increasing throughput caused an increase in latency for both mono and bi-signal queries.

4.1.2 Effect on increasing max-span:

This had a considerable difference in the impact it had on mono and bi-signal processing. For bi-signal processing, the latency increase was minimal, while it increased significantly for mono-signal. This can be explained by the fact that bi-signal doesn't have to wait for the end signal to arrive for the query, and can fire it right away, while mono-signal needs to wait for as long as the max-span is. An increase in the span directly contributes to a longer wait time.